TABLELLM: Enabling Tabular Data Manipulation by LLMs in Real Office Usage Scenarios

Xiaokang Zhang[1,3*†], **Sijia Luo**[1*], **Bohan Zhang**[1,3], **Zeyao Ma**[1,3], **Jing Zhang**[1,3‡],
Yang Li[1,3], Guanlin Li[1,3], Zijun Yao[2], Kangli Xu[2], Jinchang Zhou[2],
Daniel Zhang-Li[2], Jifan Yu[2], Shu Zhao[4], Juanzi Li[2], Jie Tang[2]

[1]School of Information, Renmin University of China, [2]Tsinghua University,
[3]Key Laboratory of Data Engineering and Knowledge Engineering, Beijing, China
[4]Computer Science, Anhui University, China

{zhang2718, luosijia0906, zbhmint, zeyaoma, zhang-jing}@ruc.edu.cn

Abstract

We introduce TABLELLM, a robust large language model (LLM) with 8 billion parameters, purpose-built for proficiently handling tabular data manipulation tasks, whether they are embedded within documents or spreadsheets, catering to real-world office scenarios. We propose a distant supervision method for training, which comprises a reasoning process extension strategy, aiding in training LLMs to understand reasoning patterns more effectively as well as a cross-way validation strategy, ensuring the quality of the automatically generated data. To evaluate the performance of TABLELLM, we have crafted benchmarks tailored to address both document and spreadsheet formats as well as constructed a well-organized evaluation pipeline capable of handling both scenarios. Thorough evaluations underscore the advantages of TABLELLM when compared to various existing general-purpose and tabular data-focused LLMs. We have publicly released the model checkpoint, source code, benchmarks, and a web application for user interaction[1].

1 Introduction

Large amounts of data are organized in tabular form and widely used in various scenarios. However, working with tabular data can be challenging, as many table-related tasks are laborious, error-prone, and require specialized skills. Automating these tasks offers significant benefits to both academic and industrial sectors, attracting considerable interest (Badaro et al., 2023; Dong et al., 2022).

To capture insights from office users, we conduct an extensive user study involving a questionnaire distributed to 507 diverse paiticipants, focusing on table-related tasks. Details about the survey are presented in Appendix C. As shown in Figure 1, participants preferred tasks involving tableQA, revision, chart creation, and matching, primarily using Excel/CSV and Word/PDF formats, including long tables. These findings highlight two characteristics of real-world office use compared to academically-focused table tasks. **(1) Diverse Operations:** user preferred tasks involve a wide range of operations, including query, update, merge, and chart, which go beyond tableQA task. **(2) Unique Processing Approaches for Different Formats:** Word/PDF documents often contain contextual textual data, allowing for hybrid querying. Excel/CSV spreadsheets contain regular and long tables, enabling more intricate operations like update and merge.

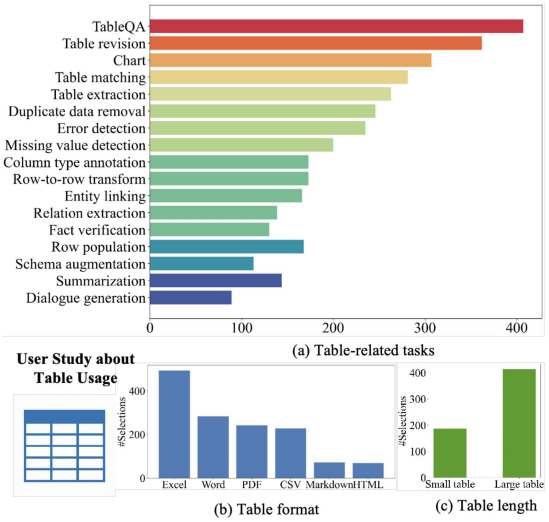


Figure 1: Illustration of the user study about (a) table-related tasks (tableQA, table revision, chart, table matching, duplicate data removal, etc.); (b) table formats (Excel, Word, etc.); (c) table length (Small: < 50 rows, Large: ≥ 50 rows).

Previous studies have focused on improving a model's reasoning capabilities for table question answering. Moving beyond tableQA, some of these endeavors have also tackled diverse table-related

*Equal Contributions.
†Work was done when interned at Zhipu AI.
‡Corresponding Author.
[1]https://tablellm.github.io/

tasks, including fact verification (Ye et al., 2023; Zhang et al., 2023a,c; Jiang et al., 2023; Liu et al., 2022), column type annotation (Li et al., 2023b; Zhang et al., 2023a), table matching (Li et al., 2023b), schema augmentation (Li et al., 2023b; Zhang et al., 2023a), and more. However, existing methods for handling tabular data in real-world office scenarios have limitations. Some use LLMs to directly extract answers from internal parameters, which is effective for document-embedded tables but inadequate for long tables and diverse spreadsheet operations. Others focus on writing and executing code for spreadsheets but struggle with hybrid queries combining text and tabular data.

Based on this insight, We present TABLELLM, specifically designed to handle a wide array of table operations in spreadsheet and document usage scenarios, named tabular data manipulation in real office usage scenarios. To facilitate model training, we introduce a distant supervision method that complements the reasoning process of existing benchmarks, aiding in training LLMs to understand reasoning patterns more effectively. Additionally, we validate the automatically generated training data through a cross-way validation strategy, ensuring data quality. We also provide a theoretical analysis of the effectiveness of cross-way validation compared to self-check and same-way validation. Utilizing this training data, we fine-tune Llama3.1(8B) (Dubey et al., 2024), resulting in the development of TABLELLM. This model adeptly handles document-embedded tabular data through an inner-parameter-driven approach and spreadsheet-embedded tabular data via a code-driven method.

A rigorous performance assessment is conducted, involving the collection of primary tableQA test instances from existing benchmarks and the creation of additional table manipulation instances by an annotation team. Given the complex evaluation process under the two scenarios, we design a meticulous evaluation method that considers query, update, merge and chart operations. **TABLELLM proves to be on par with the most capable commercial LLM GPT-4o in the document-embedded scenario, and even outperforms GPT-4o in the spreadsheet-embedded scenario.**

In the realm of tabular data processing research, our contributions encompass: (1) Addressing a practical problem of tabular data manipulation in real-world office usage scenarios. (2) Presenting techniques that extend reasoning processing and integrate a cross-way validation strategy to enhance the quality of distant supervision training data. (3) Delivering a high-quality open-source LLM tailored for tabular data manipulation in 8B, thereby enhancing accessibility and fostering collaboration within the community. (4) Offering an online application service to facilitate convenient usage.

2 Related Work

Table Question Answering. Beyond the primary tableQA task, various research endeavors tackle basic table analysis tasks such as fact verification, column type annotation and schema augmentation, typically involving web-extracted tables of relatively short length with textual content. This research has evolved through three main approaches: (1) **Representation Learning**: Many traditional methods, such as TaBERT (Yin et al., 2020), TAPAS (Herzig et al., 2020), TableGPT (Gong et al., 2020), Tabbie (Iida et al., 2021) and TAG-QA (Zhao et al., 2023), focus on intricate encoder designs with various positional encodings and attention mechanisms, while TAPEX (Liu et al., 2022) and GraPPa (Yu et al., 2020) integrate SQL execution as a pre-training task; (2) **Finetuning LLM**: Researchers leverage LLM to train unified models like TableLlama (Zhang et al., 2023a), TAT-LLM (Zhu et al., 2024) and TableGPT2 (Su et al., 2024) on multiple table-related benchmarks, with UnifiedSKG (Xie et al., 2022) extending this to structured data tasks; and (3) **Prompting LLM**: Researchers develop multi-step prompting strategies for GPT series models, employing tools like SQL and Python, such as DATER's SQL usage (Ye et al., 2023), StructGPT's self-defined interfaces (Jiang et al., 2023), and Binder's result integration method (Cheng et al., 2023).

Table Manipulation. A new research direction aims to enhance table manipulation capabilities, particularly focusing on tasks such as insert, update, and delete operations within spreadsheet formats like Excel and CSV, as well as databases (Li et al., 2023a; Dong et al., 2023; Pourreza and Rafiei, 2023; Zhang et al., 2023b). Such tasks often involve working with lengthy and regular tables, making it practical to utilize LLMs alongside tools to address them. For instance, DB-GPT (Xue et al., 2023), ChatDB (Hu et al., 2023), C3 (Dong et al., 2023) and Din-SQL (Pourreza and Rafiei, 2023) translate questions into SQL queries. SheetCopilot (Li et al., 2023a) and DataCopilot (Zhang et al.,

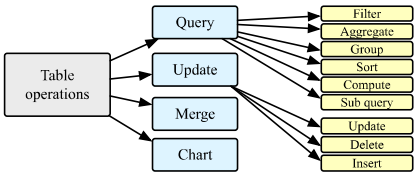


Figure 2: Common operations for table manipulation.

2023b) develop their atomic interfaces based on Excel's embedded functions and various programming languages, allowing LLMs to invoke them.

TABLELLM aims to tackle both table question answering and manipulation, making it more applicable to real-world user scenarios. Methodologically, TABLELLM fine-tunes a foundation model using synthetic data. We highlight the proposed cross-validation method, specifically designed to accommodate the unique characteristics of tables, ensuring the reliability of the synthetic data.

3 Problem Definition

Tabular Data refers to data organized in table or grid format. On top of it, **document-embedded tabular data** is tabular data integrated into documents, often in Word/PDF files, while **spreadsheet-embedded tabular data** refers to tables within spreadsheets, typically in Excel/CSV files.

Operation Definition. In light of the user study, tabular data manipulation tasks can be categorized into four primary operations: query, update, merge, and chart, as detailed in Figure 2. The "query" operation selects desired data, encompassing filter, aggregate, group, and sort functions, covering most tableQA scenarios. The "update" operation modifies, deletes or adds data, while the "merge" operation combines two tables. Lastly, the "chart" operation visualizes data using bar, pie or line charts.

Problem 1. ***Tabular Data Manipulation in Real Office Usage Scenarios*** *focuses on developing an LLM that can perform a range of query, update, merge, and chart operations with tabular data embedded in documents and spreadsheets.*

For document-embedded tabular data, querying specific information is the primary need, whereas spreadsheet-embedded tabular data often demand querying, data modification, and chart generation.

4 TABLELLM

The overview design of TABLELLM is shown in Figure 3, which consists two primary aspects: **(1) Distant Supervision Data Construction.** The development of distant supervision data involves the integration of both existing benchmark training data and new questions and answers generated from available tabular data. To enhance the training of LLMs, we suggest expanding the reasoning processes within benchmark data. Additionally, to assure the quality of the automatically generated training data, we introduce a cross-way validation strategy which utilizes diverse solution methods for cross-validation. **(2) Model Training.** The training process utilizes distinct prompts for document-embedded and spreadsheet-embedded tabular data.

4.1 Distant Supervision Data Construction

Extending Reasoning Process for Existing Benchmarks. While existing benchmarks offer ample training data for tableQA, the simple short answers provided by individual instances fall short for tackling complex tabular data manipulation tasks, which often demand intricate reasoning processes to derive answers effectively. Therefore, we augment existing benchmarks by enriching their reasoning processes to facilitate the model training.

Primarily, to address queries on document-embedded tabular data, we gather training data from widely-adopted tableQA benchmarks including WikiTQ (Pasupat and Liang, 2015), FeTaQA (Nan et al., 2022), and TAT-QA (Zhu et al., 2021). Inspired by CoT (Wei et al., 2022), We extend the provided short answers by presenting GLM-4-Plus (GLM et al., 2024) with the (question, answer) pairs and instructing it to enhance the reasoning process. This augmentation is represented in textual form, rather than as code, to align with the nature of queries involving hybrid text and tabular data inputs. Notably, for WikiTQ and FeTaQA solely provide tabular data, we supplement them by generating table descriptions using GLM-4-Plus. Due to the inner-parameter-driven technique employed, we impose a constraint on the length of input tables, limiting them to a token count of fewer than 500. To validate the quality of these text-based reasoning processes, we refer to the evaluation prompt of CritiqueLLM (Ke et al., 2023), an LLM specialized in rating, and use DeepSeek-V3 (Liu et al., 2024) to assess the consistency between the reasoning process and the answers provided in the benchmarks.

Furthermore, to handle queries on spreadsheet-embedded tabular data, we compile training

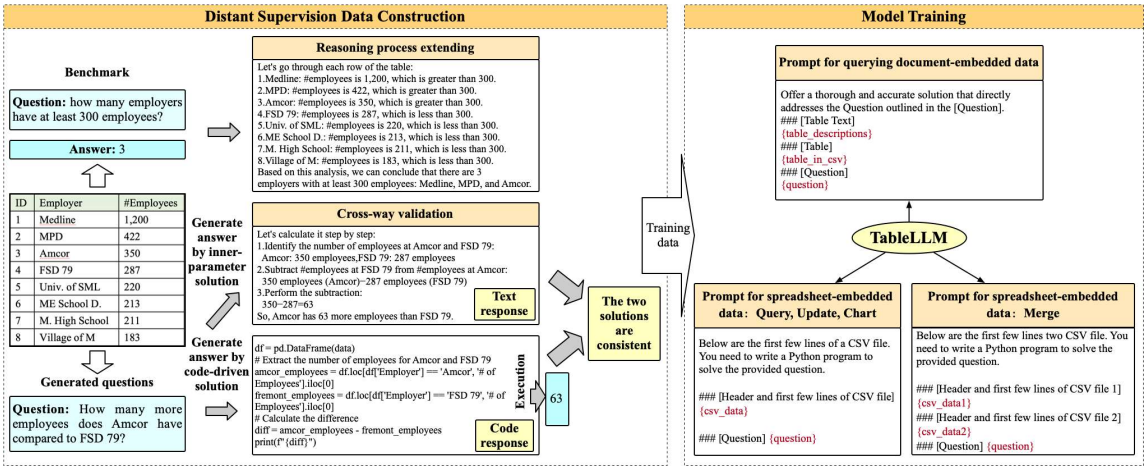


Figure 3: Overview illustration of TABLELLM. The construction of distant supervision data involves two key steps: (1) expanding the reasoning processes based on (question, answer) pairs from existing benchmarks, and (2) cross-way validation of generated (question, answer) pairs. Model training necessitates unique prompts tailored to operations in different scenarios.

data from two Text2SQL benchmarks: WikiSQL (Zhong et al., 2017) and Spider (Yu et al., 2018). Given that spreadsheet-embedded tabular data manipulation primarily involves pure tabular data inputs and complex table manipulations, it aligns more with code-driven techniques. Thus, we select training instances from WikiSQL and Spider, as they correspond to SQL queries. However, instead of directly using the provided SQL queries, we expand pandas code as the reasoning process for each (question, answer) pair by Deepseek (Bi et al., 2024), a recent powerful code LLM, as Pandas offers greater flexibility to support functionalities such as chart beyond query, update, and merge. We ensure the quality of the generated code by validating that the executed outcomes align with the provided answers in the benchmarks. Note for Spider, in line with our focus on single-table operations typical in office scenarios, we exclude multi-table queries and those whose SQL queries yield null results to better reflect real-world applications.

Automatically Generating Training Data by Cross-way Validation. While the training data derived from existing benchmarks is of high quality, the variety of questions and answers, especially the table update, merge, and chart operations they offer is limited. To address this, we introduce a cross-way validation strategy for automatically generating new questions and answers using only the provided tabular data. The process is as follows:

(1) Question Generation. We select 5,177 tables from WikiTQ, 5,000 from TAT-QA, and 4,019 from FeTaQA with less than 500 tokens to simulate document-embedded tabular data. For each table, GLM-4-Public generates questions involving single or multiple table query operations, as depicted in Figure 2. GLM-4-Public also creates contextual table descriptions for WikiTQ and FeTaQA-sourced tables, while TAT-QA tables retain their original text context. Furthermore, we select 1,300 long tables from GitTables (Hulsebos et al., 2023). For each table, we generate 10 questions involving various table manipulation operations, as illustrated in Figure 2. Existing benchmarks typically focus on table query operations, so update, chart, and merge operations are all generated. For query, update, and chart operations, we prompt GLM-4-Public for question generation. However, for the merge operation, given its well-defined nature, we directly construct templates to generate the merge question. Appendix H provides the prompts and templates used for question generation.

(2) Answer Generation and Cross-way Validation. We utilize LLMs to generate answers for the questions derived from document-embedded tables and propose a cross-validation method to ensure quality. This approach leverages the unique characteristics of tables, where answers can be obtained in two distinct ways: directly from the LLM internal parameters or by generating and executing Pandas code. By comparing the results from both methods, we ensure the reliability of the answers.

Specifically, for each question based on tabular data, we use GLM-4-Plus to generate an-

swers through both internal-parameter inference and code-driven execution. We generate 10 answers via the internal-parameter approach and 50 answers via the code-driven approach. To establish a reference answer, we aggregate the 50 code-driven answers through majority voting. Since code execution results often include additional descriptions, making exact string matching impractical, we use the ROUGE-L(Lin, 2004) metric to calculate the text similarity among the code results, cluster them, and select the centroid of the largest cluster as the reference answer. Finally, from the 10 internal-parameter-generated answers, we select the one most aligned with the reference answer as the final output. This cross-way validation, combining internal-parameter-driven and code-driven techniques, leverages their complementary strengths to enhance answer quality and reliability.

Theoretical Proof. This cross-validation approach is inspired by ensemble learning (Dietterich et al., 2002), which combines multiple weak learners to create a strong learner. Building on this concept, we conduct an improved theoretical inference to ensure the quality of automatically generated data. Let's denote Y_a as the event that the first response is correct, Y_b as the event that the second response is correct, Y as the event that both responses are correct, and E as the event that the two responses are consistent. Based on these definitions, we can establish the following theorem:

Theorem 4.1. *(1) If A and B are drawn from the same distribution such that $P(Y_a) = P(Y_b) = p > 1/2$, then consistency checking outperforms single inference, i.e., $P(Y|E) \geq P(Y_a)$. (2) If A and B are further drawn from independent distributions, the effect will be superior (in terms of expectation).*

This theorem suggests that when the model's probability of providing correct answers exceeds 1/2, employing consistency verification is decisively more effective than direct inference. Moreover, in terms of expected performance, utilizing cross-validation with two independent distributions surpasses consistency checks with a single distribution. The proof is provided in Appendix D.

For questions on spreadsheet-embedded tabular data, we employ GLM-4-Plus to generate a code solution, which is followed by the generation of an alternative code solution using GLM-4-Plus again. The accuracy of the executed outcomes from the first code are verified by comparing them with the outcomes of the second code. Given the potential diversity of two coding solutions yielding the same answers, this dual-coding strategy can be regarded as stemming from different distributions. Thus it also functions as a cross-way validation method.

4.2 Model Training

In the scenario of document-embedded tabular data, the input for LLMs includes both the text and the entire content of the table. However, in the case of spreadsheet-embedded tabular data, due to the typically extensive length of the table, only the header and a subset of rows are provided as input to the LLM. The prompt for the merge operation is specifically designed, illustrated in Figure 3.

Given the prompt x as input, we enable LLMs to generate either the textural or code solution, collectively denoted as y. We hybridize the document-embedded and spreadsheet-embedded training data in a 1:1 ratio, thoroughly shuffle them, and then partition them into batches for training.

The trained single model addresses two types of data sources. Given that code models tend to excel in reasoning-intensive tasks compared to text models (Liang et al., 2023), combining the two data sources can enhance text-level reasoning with code-level reasoning. Moreover, a single model could alleviate deployment pressure.

4.3 Model Deployment as Web Application

We launch our TABLELLM as a web application, with a screenshot shown in Figure 5. Users can upload tabular data from documents (Word, PDF) and spreadsheets (Excel, CSV). The system parses PDF and Word files into CSV for visualization. Users enter queries, and the TABLELLM generates responses—tables, charts, or text—based on prompts and document type. It also supports table merging, allowing users to merge two spreadsheets with specified conditions[2]. Details are in Appendix B.

5 Experiment

5.1 Test Set Creation

We collect test sets from established benchmarks for document-embedded query tasks, including WikiTQ (Pasupat and Liang, 2015), FeTaQA (Nan et al., 2022), and TAT-QA (Zhu et al., 2021). For spreadsheet-embedded table tasks, we utilize test sets from WikiSQL (Zhong et al., 2017) and Spider (Yu et al., 2018), which align with our query

[2]Currently, the system is configured to support the merging of two tables only.

Table 1: Benchmark (test set) statistics

Scenario	Name	Description	Size
Document -embedded	WikiTQ	<500 tokens & add text	633
	FeTaQA	<500 tokens & add text	753
	TAT-QA	<500 tokens	800
Spreadsheet -embedded	WikiSQL	Remove vague questions	1,000
	Spider	Choose single table	512
	Our created	Query/Update/Merge/Chart	1,200
Both	TABLELLM-bench	-	4,898

operation requirements. We extract the table, question, and answer from each instance.

As no benchmarks exist for table update, merge, and chart operations, we create test set through human annotation. We choose 50 long tables from InfiAgent-DABench (Hu et al., 2024), ensuring they are entirely distinct from our training data. Following the process outlined in Section 4.1, we generate questions and answers. An annotator team verifies the generated content, including answers, codes, and operation types. Since initial questions lack linguistic diversity, we utilize five prominent models from Huggingface for rewriting to enhance variety. This results in a composition of 10% original questions and 90% rewritten ones, with each model contributing to 18% of the rewrites. Table 1 displays the benchmark statistics.

5.2 Evaluation Approach

Given the diverse range of operation types in our dataset, we have adopted a categorized evaluation approach across different operations:

- **Query operations:** For the answers obtained through code execution, we conduct an exact match comparison between the model's output and the ground truth answers. For answers inferred directly via inner-parameters, we use DeepSeek-V3 to assign a score from 1 to 10 by comparing the model's output with the ground truth answers, with a score threshold of 7 considered correct, as discussed in Section 4.1. This is because the generated answers are often lengthy and challenging to precisely match. We also conduct a meta evaluation on DeepSeek-V3's rating scores by humans, as detailed in Appendix J.
- **Update and merge operations:** As these operations directly modify tables, we require the model's output to be the complete modified table. We then perform an exact match comparison between the model's output and the ground truth answers to determine correctness.
- **Chart operations:** Assessing charting operations is challenging through direct answer comparison. Instead, we compare model code output with ground truth code. DeepSeek-V3 is once again employed to compare the model's output code with the ground truth code, using a score threshold of 5 for evaluation.

Based on the correctness determination, we assess accuracy.

5.3 Comparison Methods

Comparison methods are divided into four types:

- **Pre-training and fine-tuning LLMs:** This category encompasses models like TaPas (Herzig et al., 2020) (based on BERT) TAPEX (Liu et al., 2022) (based on BART), TableLlama (Zhang et al., 2023a) (based on Llama2 (7B)) and TableGPT2(7B) (Su et al., 2024).
- **General LLMs:** This group includes GPT-3.5 (Ouyang et al., 2022), GPT-4o (OpenAI, 2023), and Llama3.1 (8B) (Dubey et al., 2024).
- **Coding-specific LLMs:** This category contains LLMs tailored for coding tasks, including CodeLlama (Rozière et al., 2023) and DeepSeek (Bi et al., 2024).
- **Prompt-driven LLMs:** This group includes StructGPT (Jiang et al., 2023), ReAcTable (Zhang et al., 2023c), Binder (Cheng et al., 2023), and DATER (Ye et al., 2023), focusing on creating sophisticated prompts to guide LLMs in processing tabular data.

Implementation. (1) TaPas and TAPEX have individual checkpoints trained on WikiTQ and WikiSQL. We assess their performance in document-embedded tabular data scenarios using the WikiTQ-trained versions and in spreadsheet-embedded tabular data scenarios using the WikiSQL-trained versions. As for TableLlama and TableGPT2, we evaluate its unique checkpoints directly. (2) For both general and coding-specific LLMs, we provide customized prompts for scenarios to handle document-embedded and spreadsheet-embedded tabular data, as detailed in Appendix I. (3) Prompt-driven LLMs follow their established prompts. We unify StructGPT's prompts for WikiTQ, TAT-QA, FeTaQA, and WikiSQL to match WikiTQ's format. Meanwhile, Binder and DATER use a single unified set of prompts across all benchmarks. (4) TABLELLM is trained using Llama3.1 (8B). During inference, we consistently apply the same set of prompts used during training phase. The generated training data is presented in Table 7 in Appendix G.

Table 2: Overall evaluation in both document-embedded and spreadsheet-embedded tabular data scenarios (%)

Model	Document-embedded tabular data			Spreadsheet-embedded tabular data			Average accuracy	Inference times
	WikiTQ	TAT-QA	FeTaQA	WikiSQL	Spider	Our created		
TaPEX	38.55	–	–	83.90	15.04	–	45.83	1
TaPas	31.60	–	–	74.20	23.05	–	42.95	1
TableLlama	24.01	22.25	20.47	43.70	–	–	23.36	1
TableGPT2(7B)	77.25	88.12	75.58	63.0	77.34	74.42	75.95	1
Llama3.1(8B)	71.88	74.25	83.40	40.60	18.75	43.17	55.34	1
GPT-3.5	58.45	72.13	71.18	81.70	67.38	77.08	69.82	1
GPT-4o	**91.47**	**91.50**	**94.42**	<u>84.00</u>	69.53	<u>77.83</u>	<u>84.79</u>	1
CodeLlama (13B)	43.44	47.25	57.24	38.30	21.88	47.58	43.63	1
Deepseek-Coder (33B)	6.48	11.00	7.12	72.50	58.40	73.92	33.84	1
StructGPT (GPT-3.5)	52.45	27.53	11.80	67.80	**84.80**	–	43.06	3
Binder (GPT-3.5)	61.61	12.77	6.85	78.60	52.55	–	36.25	50
DATER (GPT-3.5)	53.40	28.45	18.26	58.20	26.52	–	32.98	100
TABLELLM (8B)	<u>89.10</u>	<u>89.50</u>	<u>93.36</u>	**89.6**	<u>81.05</u>	<u>77.83</u>	**86.74**	1

* Underline represents the runner up.

5.4 Overall Experimental Results

Effectiveness. Table 2 displays the overall evaluation in two scenarios. "–" in the table indicates that the method does not support the dataset or that the tested accuracy is too low. The results show that **TABLELLM generally surpasses others in the spreadsheet-embedded scenario and is on par with GPT-4o in the document-embedded scenario**. Detailed findings include:

(1) TaPEX and TaPas show limited performance due to their small model sizes. These two pre-training and fine-tuning models only demonstrate relatively strong performance on WikiSQL and WikiTQ benchmarks.

(2) StructGPT, Binder, and DATER's varying performance across datasets suggests a limitation in the generalization capability of prompt-driven LLMs. While these models consistently perform well in the WikiTQ benchmark, their performance weakens on other datasets. StructGPT stands out in the Spider benchmark due to its customized prompts tailored for this specific dataset.

(3) DeepSeek (33B) excels in the spreadsheet-embedded tabular data scenario. This superior performance is attributed to its extensive optimization for coding capabilities. However, this specialization in coding comes at the expense of direct answer inference from inner parameters when dealing with document-embedded tabular data.

(4) Our TABLELLM outperforms GPT-3.5 and GPT-4o in the spreadsheet-embedded scenario. Moreover, in our created benchmark with entirely distinct tabular data and questions from the training data, TABLELLM achieves an impressive 77.83% accuracy, showcasing robust generalization ability. Conversely, in the document-embedded scenario, TABLELLM performs close to GPT-4o and significantly better than GPT-3.5, possibly due to the scenario's demand for extensive commonsense reasoning with text data, where TABLELLM could benefit from enhanced training in text QA.

Table 3: Effect of diverse training data sources (%)

Train data	Document-embedded		Spreadsheet-embedded	
	WikiTQ	TAT-QA	Spider	Our created
Llama3.1 (8B)	71.9	74.3	18.8	43.2
Original train data	71.3	68.5	–	–
Extended train data	83.1	87.5	77.9	55.8
Generated train data	82.2	86.3	62.1	72.2
Mixed data	84.0	88.3	80.9	73.6

Table 4: Effect of cross-way validation strategy on document-embedded tabular data (%)

Validation strategy	WikiTQ	TAT-QA
Self-check validation	74.5	81.3
Same-way validation	77.8	85.8
Cross-way validation	84.7	87.9

Efficiency. All methods, except prompt-driven LLMs, require only one inference process per instance. However, Binder necessitates a one-step inference for each instance, requiring 50 samples per step for self-consistency validation. DATER requires four-step inferences for each instance, with self-consistency validation at each step, totaling 100 inferences per instance. StructGPT requires three inferences per question.

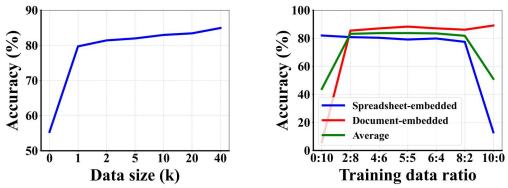
5.5 Ablation Studies on Training Data

Effect of Diverse Training Data Sources. We analyze the influence of different training datasets by comparing five distinct training configurations:

- **Llama3.1 (8B)**: The base version without any training.
- **With original training data of existing benchmarks**: Train Llama3.1 using 2,000 training instances from TAT-QA and WikiTQ, then evaluate on corresponding test sets.
- **With extended training data of existing benchmarks**: Train on 2,000 training instances from WikiTQ and TAT-QA, supplemented with extended reasoning process. Train on 2,000 training instances from Spider, supplemented with extended code, and evaluate on both Spider and our created test sets.
- **With generated training data**: Train on 2,000 generated instances based on WikiTQ/TAT-QA's tabular data, then test on corresponding test sets. Train on 2,000 generated code-outputted instances based on GitLab's tabular data, and evaluate on Spider and our created test sets.
- **With mixed data**: Train with a mix of 2,000 extended and 2,000 generated instances from TAT-QA/WikiTQ, then evaluate on corresponding test sets. Train with a mix of 2,000 extended Spider training instances and 2,000 generated code-outputted instances, and evaluate on both Spider and our created test sets.

As shown in Table 3, including the original training data leads to worse performance than the base version without training. This may be because the earlier benchmark data is too simplistic, hindering LLM training. The results demonstrate the effectiveness of incorporating extended reasoning processes, showcasing a performance boost of 15.6% and 17.8% on WikiTQ and TAT-QA respectively, compared to the base version. This improvement is mainly due to the inclusion of detailed explanations, helping LLMs recognize reasoning patterns. Furthermore, the addition of generated data yields an additional 14.3% and 16.2% enhancement in performance over the base version on WikiTQ and TAT-QA, respectively, emphasizing the value of automatically generated data. Notably, the combination of extended and generated training data leads to a significant 16.8% and 18.8% increase in performance relative to the base version, highlighting the advantages of integrating diverse data sources. The results observed on Spider and our created test sets further corroborate the benefits of extended and generated training data.

Effect of Cross-way Validation. We examine the effectiveness of cross-way validation methods on document-embedded tabular data. For fair comparison, our cross-way validation method derives one answer from both the inner-parameter solution and the code-driven solution. We compare it against two other validation methods: **Same-way validation**, which generates two direct answers by the same inner-parameter technique of GLM-4-Plus and assesses their alignment using DeepSeek-V3, and **Self-check validation**, which enables GLM-4-Plus to generate one textual solution and self-check its answer. Table 4 presents that our cross-way validation method outperforms the other methods, due to its use of two distinct responses.



(a) Data size (b) Data ratio

Figure 4: Effects of data size and ratio.

5.6 Training Strategy Investigation

We investigate two training aspects: data size and the ratio between document- and spreadsheet-embedded data. We explore the following variants:

- **Data size:** Options include 1k, 2k, 5k, 10k, 20k, and 40k instances.
- **Data ratio:** The proportion of document- to spreadsheet-embedded data, explored in ratios of 0:10, 2:8, 4:6, 5:5, 6:4, 8:2, and 10:0.

The default configuration for our experiments is 10k training data instances, a 5:5 data ratio.

Figure 4 shows TABLELLM 's accuracies under various training data settings. As depicted in Figure 4(a), performance gains follow a log-linear relationship with the training data size, motivating us to stop early at 40K, which offers a cost-effective balance. Figure 4(b) indicates that a 5:5 ratio yields balanced performance across both data types.

6 Conclusion

This paper introduces TABLELLM (8B) tailored for tabular data manipulation in real office scenarios. We gather actual requirements from office settings and identify document-embedded and spreadsheet-embedded scenarios. Ensuring high-quality data through extended reasoning processes and cross-way validation on automatically gener-

ated training data, the resulting TABLELLM performs comparably to GPT-4o and surpasses it in the spreadsheet-embedded scenario. We anticipate that our dataset, model checkpoint, and code will provide a cost-effective solution for enhancing LLM capabilities for tables.

Limitations

Diversity of Generated Data. The training data, although extensive, may not fully capture the diversity of tabular data encountered in real-world applications. In the question generation phase of the cross-way strategy, the questions generated by GLM-4-Public are based on tabular data from the existing benchmark, which may not cover all the cases, and some of the generated questions are not challenging enough to enhance the model's reasoning about the tabular data. Therefore, the reliance on existing benchmarks and automatically generated data could introduce biases or gaps in the model's understanding of less common or more complex table structures. Future research could further improve the quality of synthetic training data by categorizing the difficulty of generating problems.

References

Gilbert Badaro, Mohammed Saeed, et al. 2023. Transformers for tabular data representation: A survey of models and applications. *TACL*.

Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*.

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding language models in symbolic languages. *ICLR*.

Thomas G Dietterich et al. 2002. Ensemble learning. *The handbook of brain theory and neural networks*, 2(1):110–125.

Haoyu Dong, Zhoujun Cheng, et al. 2022. Table Pre-training: A Survey on Model Architectures, Pre-training Objectives, and Downstream Tasks. In *IJCAI*.

Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: zero-shot text-to-sql with chatgpt. *CoRR*, abs/2307.07306.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*.

Heng Gong, Yawei Sun, Xiaocheng Feng, Bing Qin, Wei Bi, Xiaojiang Liu, and Ting Liu. 2020. TableGPT: Few-shot table-to-text generation with table structure reconstruction and content matching. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 1978–1988, Barcelona, Spain (Online). International Committee on Computational Linguistics.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333, Online. Association for Computational Linguistics.

Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. Chatdb: Augmenting llms with databases as their symbolic memory. *arXiv preprint arXiv:2306.03901*.

Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. 2024. Infiagent-dabench: Evaluating agents on data analysis tasks. *Preprint*, arXiv:2401.05507.

Madelon Hulsebos, Çagatay Demiralp, and Paul Groth. 2023. Gittables: A large-scale corpus of relational tables. *Proceedings of the ACM on Management of Data*, 1(1):1–17.

Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. 2021. Tabbie: Pretrained representations of tabular data. *arXiv preprint arXiv:2105.02584*.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9237–9251, Singapore. Association for Computational Linguistics.

Pei Ke, Bosi Wen, Zhuoer Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, Jie Tang, and Minlie Huang. 2023. Critiquellm: Scaling llm-as-critic for effective and explainable evaluation of large language model generation. *Preprint*, arXiv:2311.18702.